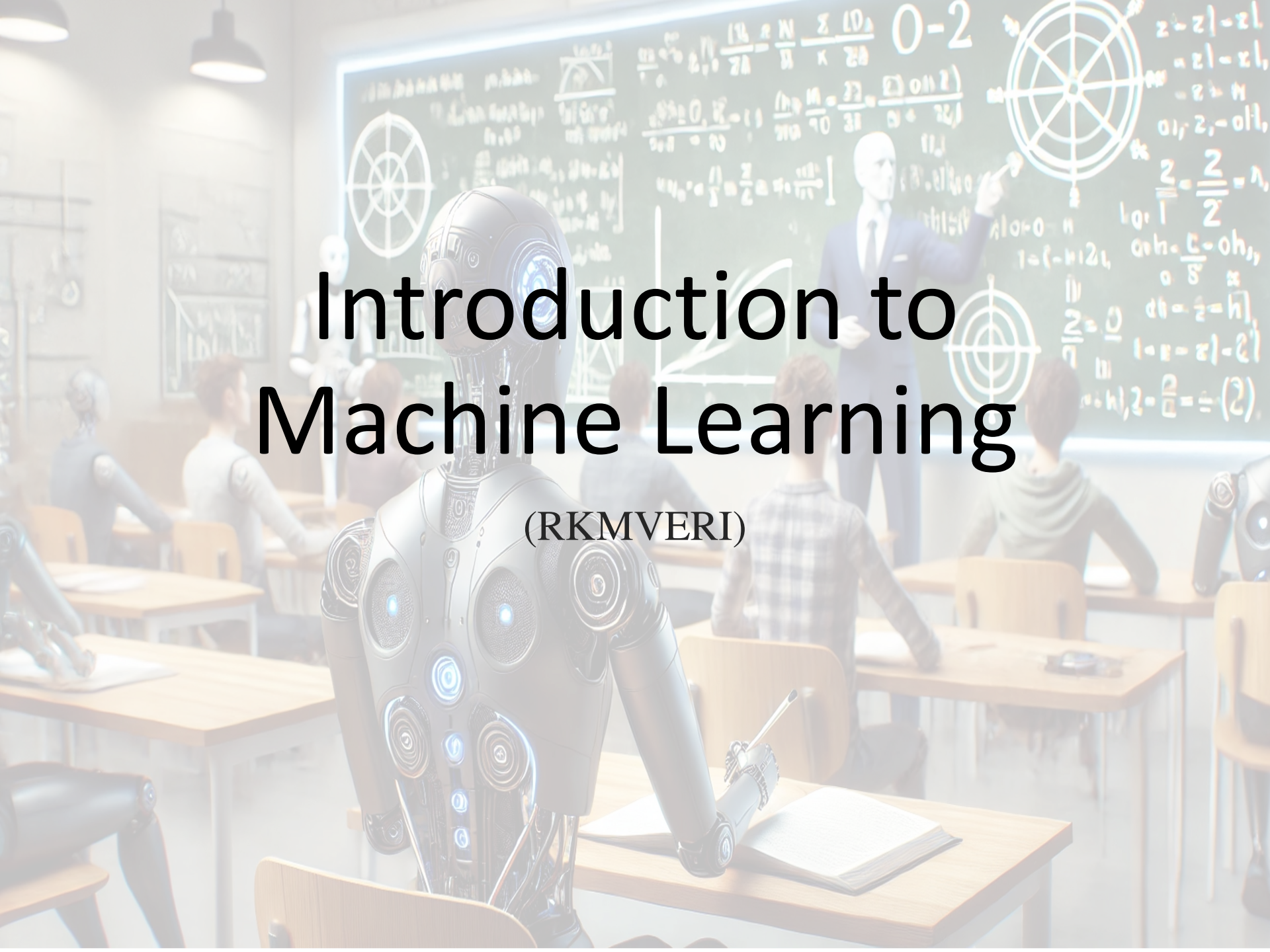


A futuristic classroom scene. In the foreground, a robot student with a metallic body and glowing blue lights is sitting at a wooden desk, holding a pen over an open book. In the background, a teacher robot in a suit is pointing at a chalkboard filled with complex mathematical formulas and diagrams. Other students, some human and some robotic, are seated at desks in the background. The overall atmosphere is one of advanced education and technology.

Introduction to Machine Learning

(RKMVERI)

Shanti Mantra

ॐ सह नावतु ।

सह नौ भुनक्तु ।

सह वीर्यं करवावहै ।

तेजस्वि नावधीतमस्तु मा विद्विषावहै ।

ॐ शान्तिः शान्तिः शान्तिः ॥

Om Saha Nau-Avatu |

Saha Nau Bhunaktu |

Saha Viiryam Karavaavahai |

Tejasvi Nau-Adhiitam-Astu Maa Vidvissaavahai |

Om Shaantih Shaantih Shaantih ||

Om, may He protect us both (the teacher and the student).

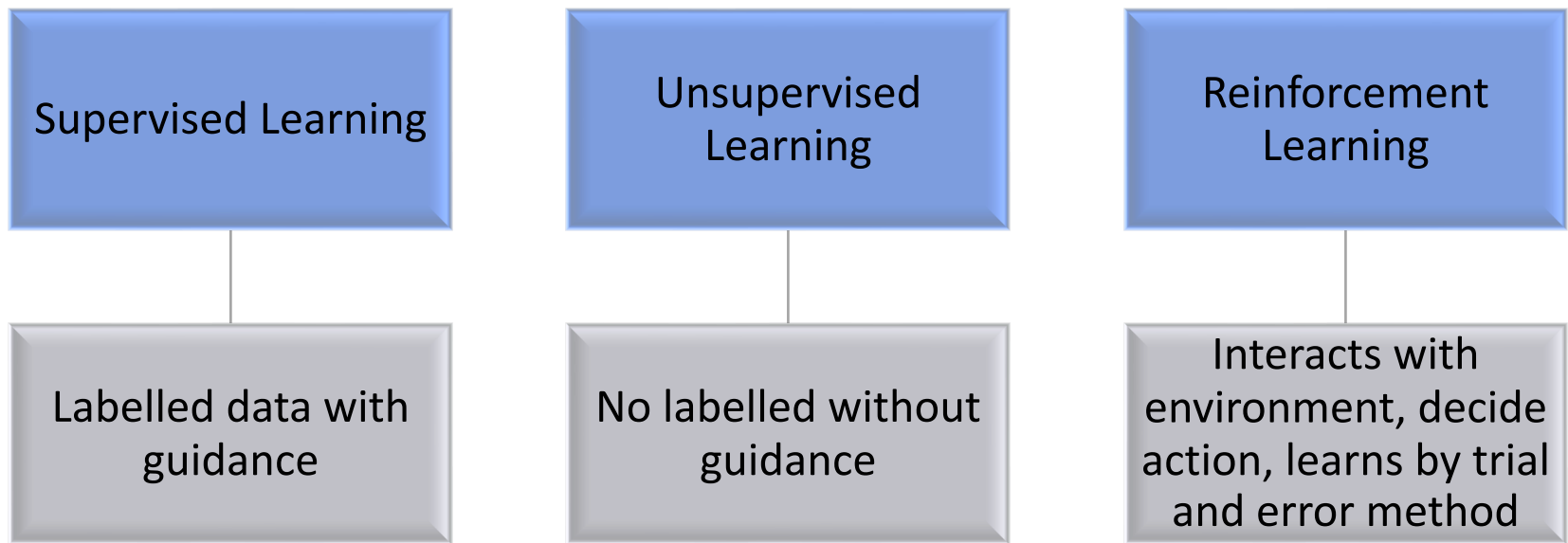
May He nourish us both.

May we work together with energy and vigor.

May our study be enlightening and not give rise to hostility.

Om, peace, peace, peace.

AI Techniques



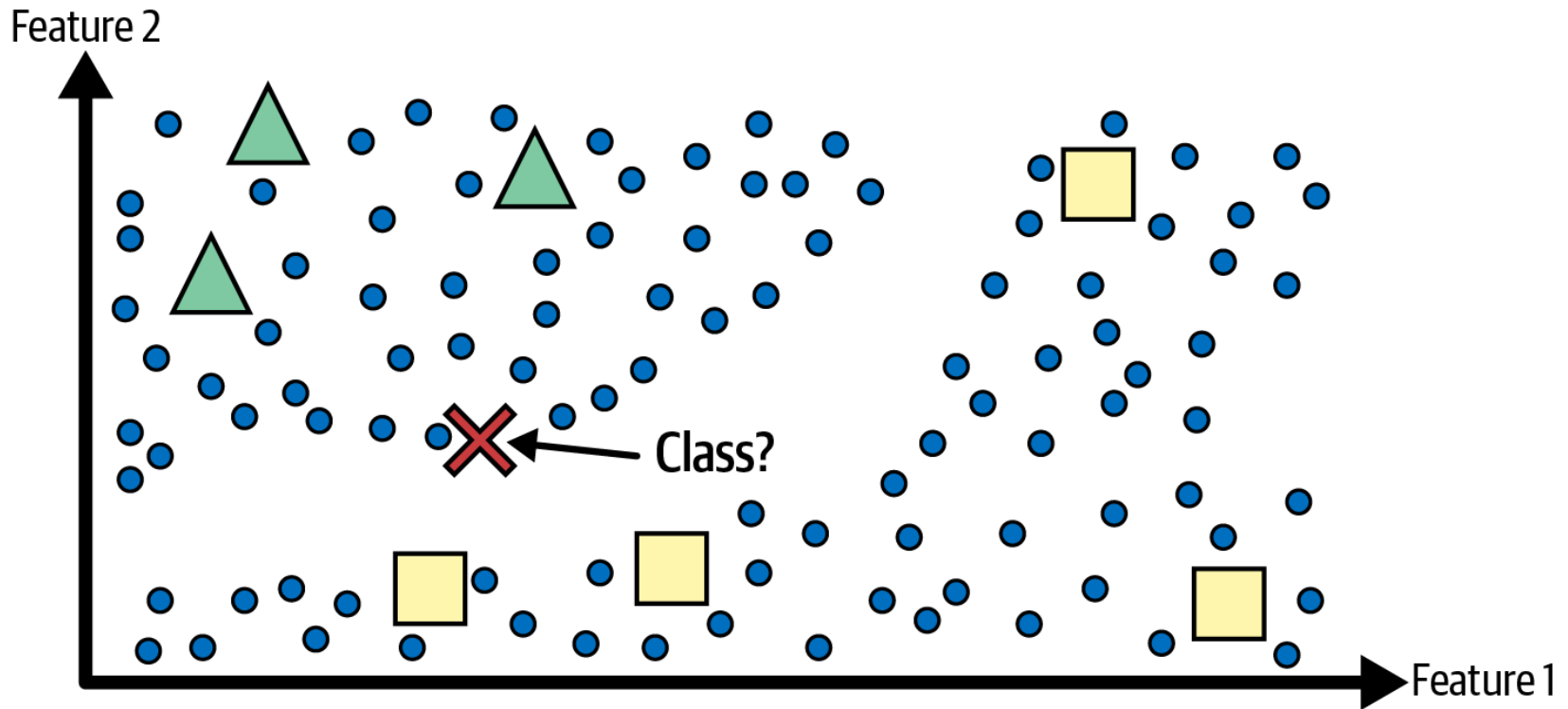
What about ChatGPT

- ChatGPT and similar models fall under the category of Supervised Learning, specifically within the subfield of Natural Language Processing (NLP).

Where does ChatGPT fit?

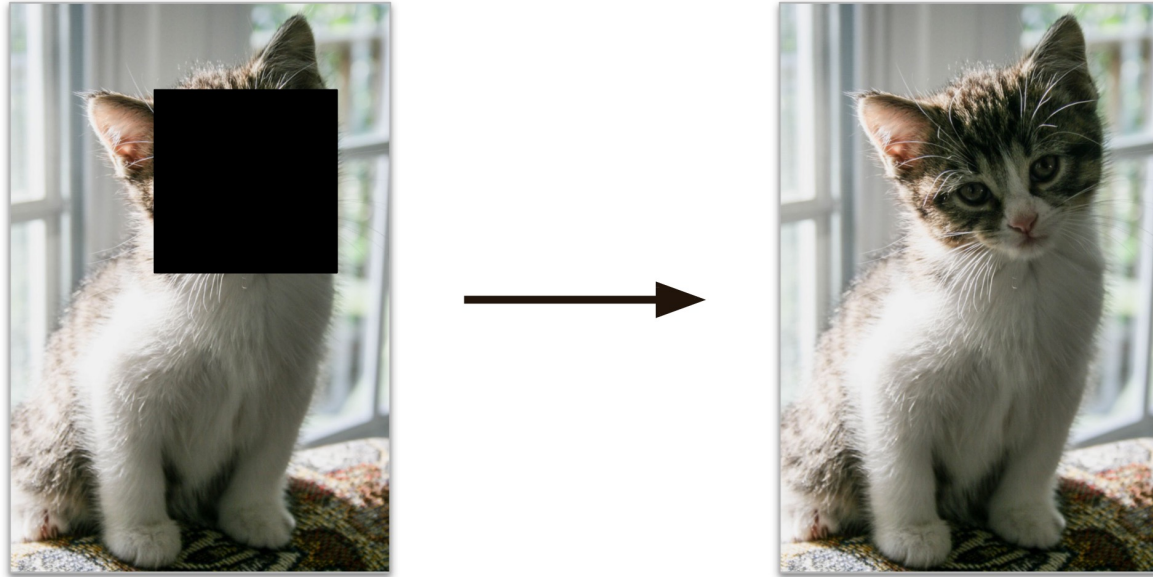
- ChatGPT and similar models fall under the category of **Supervised Learning**, specifically within the subfield of Natural Language Processing (NLP).
- **Pre Training**: pre-trained on a large corpus of text data using a task known as language modeling.
- **Fine-Tuning**: further trained on a narrower dataset with explicit labels or specific tasks.
 - E.g., on dialogue datasets where it learns to generate appropriate responses
- **NOTE: Reinforcement Learning can be employed**

Semi-supervised learning



- Semi-supervised learning with two classes (triangles and squares):
- The unlabeled examples (circles) help classify a new instance (the cross) into the triangle class rather than the square class, even though it is closer to the labeled squares

Self-supervised learning



Self-supervised learning example: input (left) and target (right)

If you have a large dataset of unlabeled images, you can randomly mask a small part of each image and then train a model to recover the original image

During training, the masked images are used as the inputs to the model, and the original images are used as the labels.

Batch vs. Online Learning

Another criterion used to classify machine learning systems is whether or not the system can learn incrementally from a stream of incoming data.

Batch learning

- The system must be trained using all the available data.
- Generally takes a lot of time and computing resources, so it is typically done offline.
- First the system is trained, and then it is launched into production and runs without learning anymore; it just applies what it has learned.
- This is called offline learning.
- Cons: Model's performance tends to decay slowly over time. This phenomenon is often called model rot or data drift.
- Solution: Regularly retrain the model on up-to-date data.

Online Learning

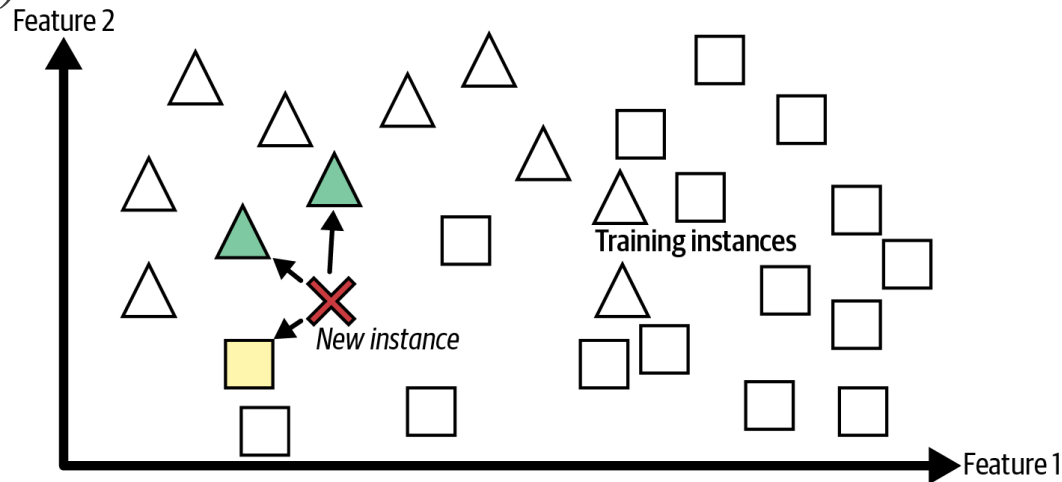
- The system is trained incrementally by feeding it data instances sequentially, either individually or in small groups called mini-batches.
- Each learning step is fast and cheap, so the system can learn about new data on the fly, as it arrives
- can be used to train models on huge datasets that cannot fit in one machine's main memory (this is called out-of-core learning).
- The algorithm loads part of the data, runs a training step on that data, and repeats the process until it has run on all of the data

Instance-Based vs. Model-Based Learning

- One more way to categorize machine learning systems is by how they generalize.
- Given a number of training examples, the system needs to be able to make good predictions for (generalize to) examples it has never seen before (generalization).
- Having a good performance measure on the training data is good, but insufficient; the true goal is to perform well on new instances.
- Two main approaches to generalization:
- Instance-based learning and Model-based learning.

Instance-Based Learning

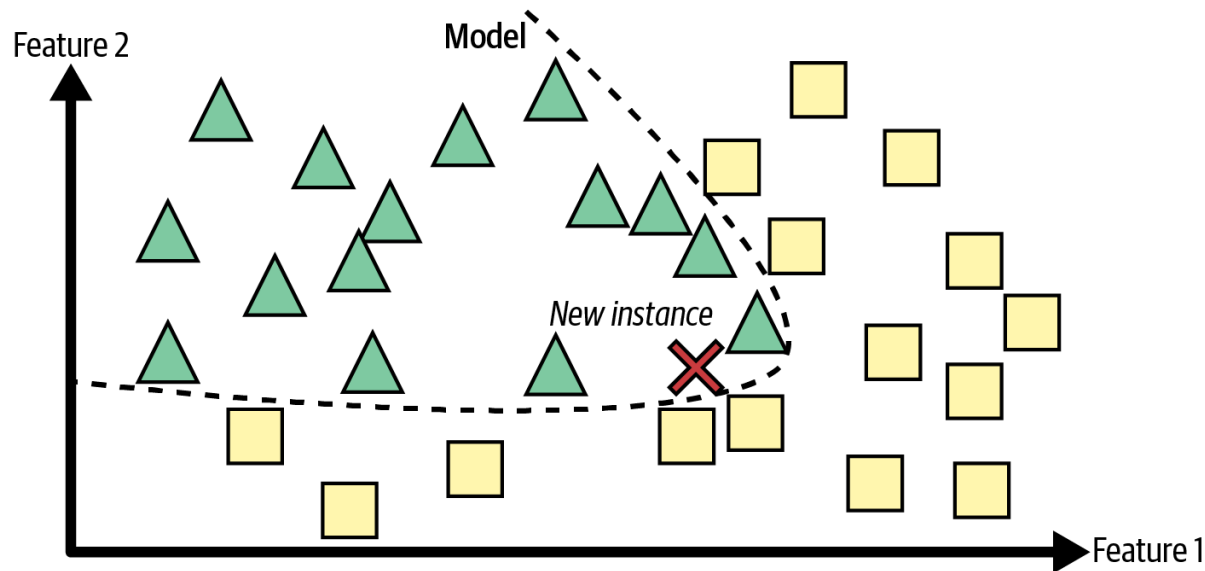
- The system learns the examples by heart, then generalizes to new cases by using a similarity measure to compare them to the learned examples (or a subset of them).



- For example, in the figure the new instance would be classified as a triangle because the majority of the most similar instances belong to that class.

Model-based Learning

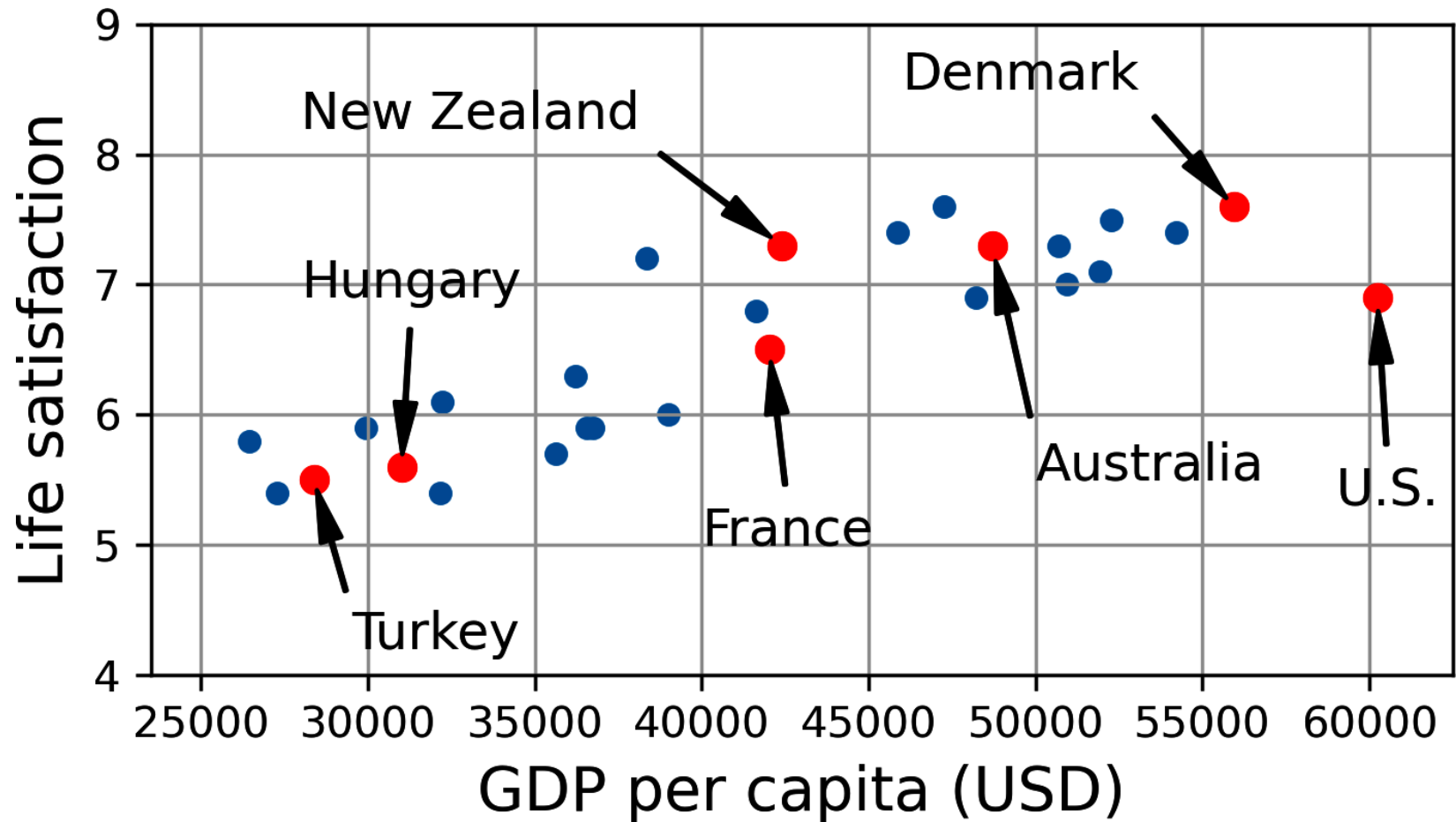
- Another way to generalize from a set of examples is to build a model of these examples and then use that model to make predictions.
- This is called model-based learning



Does money make people happier?

Country	GDP per capita (USD)	Life satisfaction
Turkey	28,384	5.5
Hungary	31,008	5.6
France	42,026	6.5
United States	60,236	6.9
New Zealand	42,404	7.3
Australia	48,698	7.3
Denmark	55,938	7.6

Do you see a trend here?



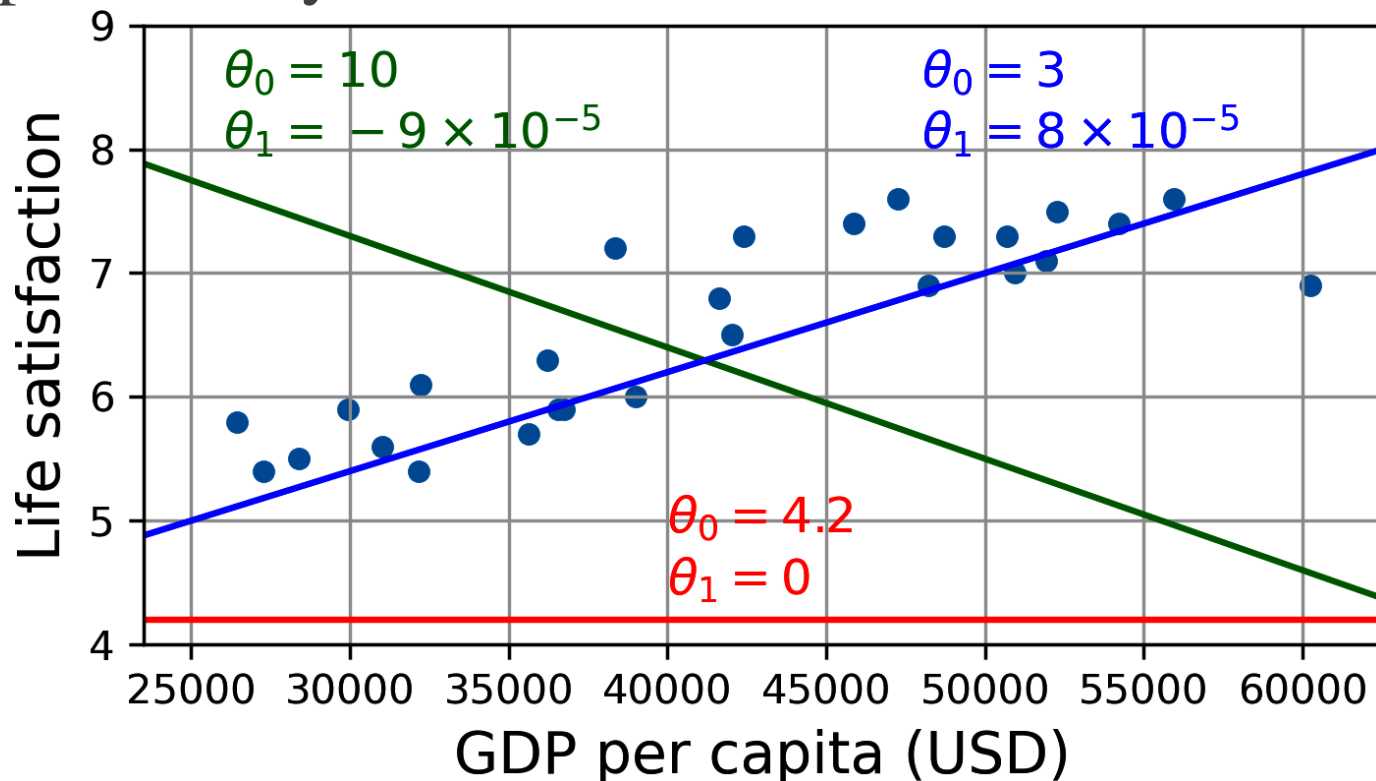
There does seem to be a trend here!

- Although the data is noisy (i.e., partly random), it looks like life satisfaction goes up more or less linearly as the country's GDP per capita increases.
- So you decide to model life satisfaction as a linear function of GDP per capita.
- This step is called **model selection**: you selected a linear model of life satisfaction with just one attribute, GDP per capita

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$

Model Parameters

- Two model parameters: θ_0 and θ_1
- By tweaking these parameters, the model can represent any linear function



Model Performance

- How can you know which values will make your model perform best?
- Need to specify a performance measure:
 - **Utility function (or fitness function)** that measures how good your model is, or you can define
 - **Cost function** that measures how bad it is.
- For linear regression problems, people typically use a cost function that measures the distance between the linear model's predictions and the training examples
- The objective is to minimize this distance.

Linear Regression

- you feed it your training examples, and it finds the parameters that make the linear model fit best to your data.
- This is called **training the model**.
- In our case, the algorithm finds that the optimal parameter values are

$$\theta_0 = 3.75 \quad \text{and} \quad \theta_1 = 6.78 \times 10^{-5}$$

CODE

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Download and prepare the data
data_root = "https://github.com/ageron/data/raw/main/"
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

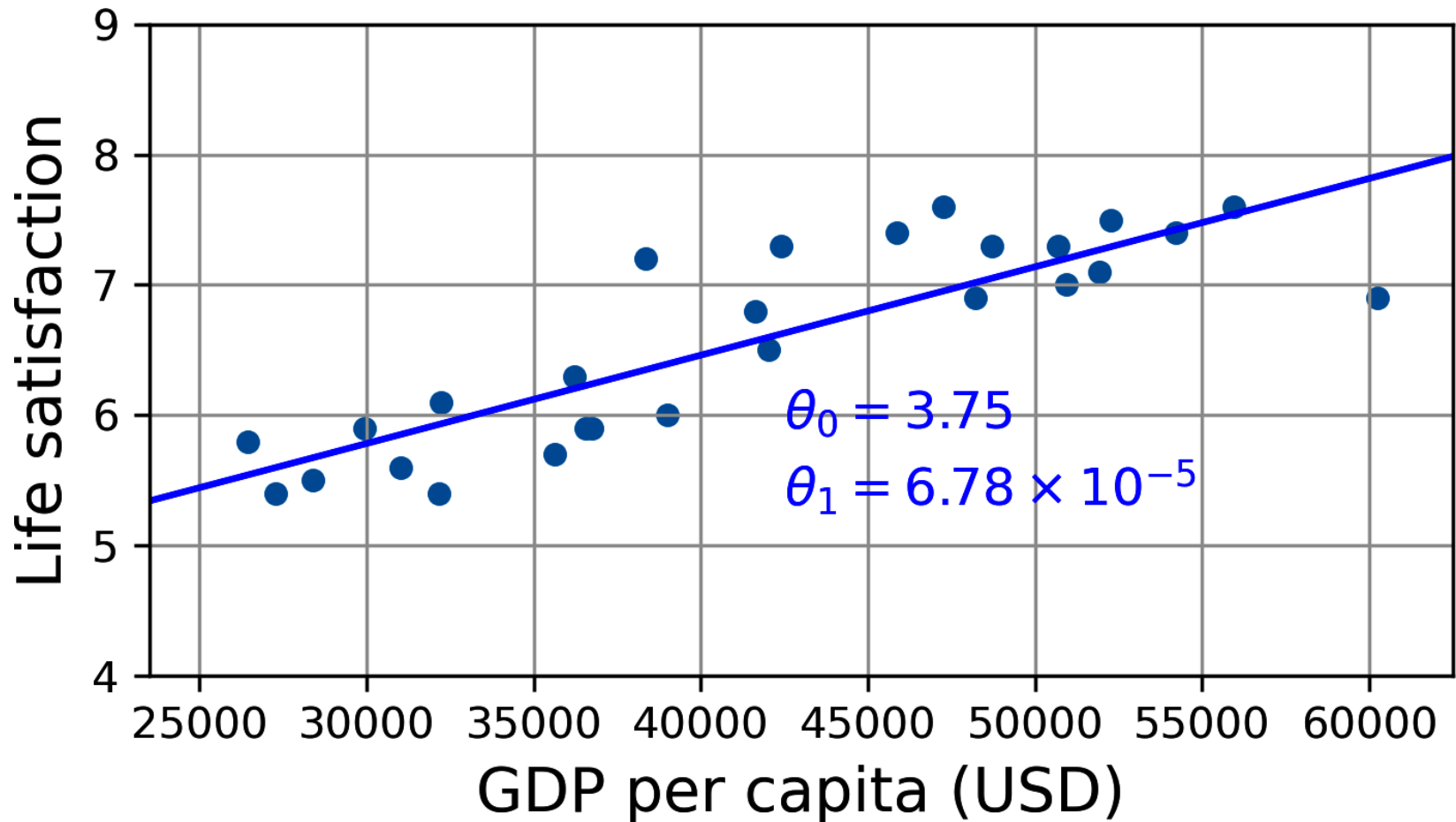
# Visualize the data
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# Select a linear model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Make a prediction for Cyprus
X_new = [[37_655.2]] # Cyprus' GDP per capita in 2020
print(model.predict(X_new)) # output: [[6.30165767]]
```

The linear model that fits the training data best



“Model” can refer to:

- A type of model (e.g., linear regression)
- A fully specified model architecture (e.g., linear regression with one input and one output), or
- Final trained model ready to be used for predictions (e.g., linear regression with one input and one output, using

$$\theta_0 = 3.75 \quad \text{and} \quad \theta_1 = 6.78 \times 10^{-5}$$

- Model selection consists in choosing the type of model and fully specifying its architecture.
- Training a model means running an algorithm to find the model parameters that will make it best fit the training data, and hopefully make good predictions on new data.

Predictions

- You have to predict how happy **Cypriots** are, but the data is not present.
- Fortunately, you can use your model to make a good prediction:
- **Given that Cyprus's GDP per capita = \$37,655**
- So, **life satisfaction** is likely to be somewhere around

$$3.75 + 37,655 \times 6.78 \times 10^{-5} = 6.30$$

Instance Based Learning

- Israel has the closest GDP per capita to that of Cyprus (\$38,341),
- Data tells us that Israelis' life satisfaction is **7.2**
- So, predicted life satisfaction of Cyprus would be **7.2**
- The two next-closest countries, Lithuania and Slovenia, both with a life satisfaction of **5.9**.
- Averaging these three values, we get **6.33**, which is pretty close to the model-based prediction.
- This simple algorithm is called **k-nearest neighbors regression** (in this example, $k = 3$).

CODE

Replacing the linear regression model with k -nearest neighbors regression in the previous code is as easy as replacing these lines:

```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()
```

with these two:

```
from sklearn.neighbors import KNeighborsRegressor  
model = KNeighborsRegressor(n_neighbors=3)
```

A typical ML project

- Study the data
- Select a Model
- Train the model on the training data
- Finally, make predictions on new cases
 - this is called inference

Main Challenges of ML

- Since the main task is to select a model and train it on some data, the two things that can go wrong are

1. “bad model”

2. “bad data”.

Insufficient Quantity of Training Data

- It takes a lot of data for most ML algorithms to work properly.
- Even for very simple problems we typically need thousands of examples, and
- For complex problems such as image or speech recognition you may need millions of examples (unless you can reuse parts of an existing model).

The Unreasonable Effectiveness of Data

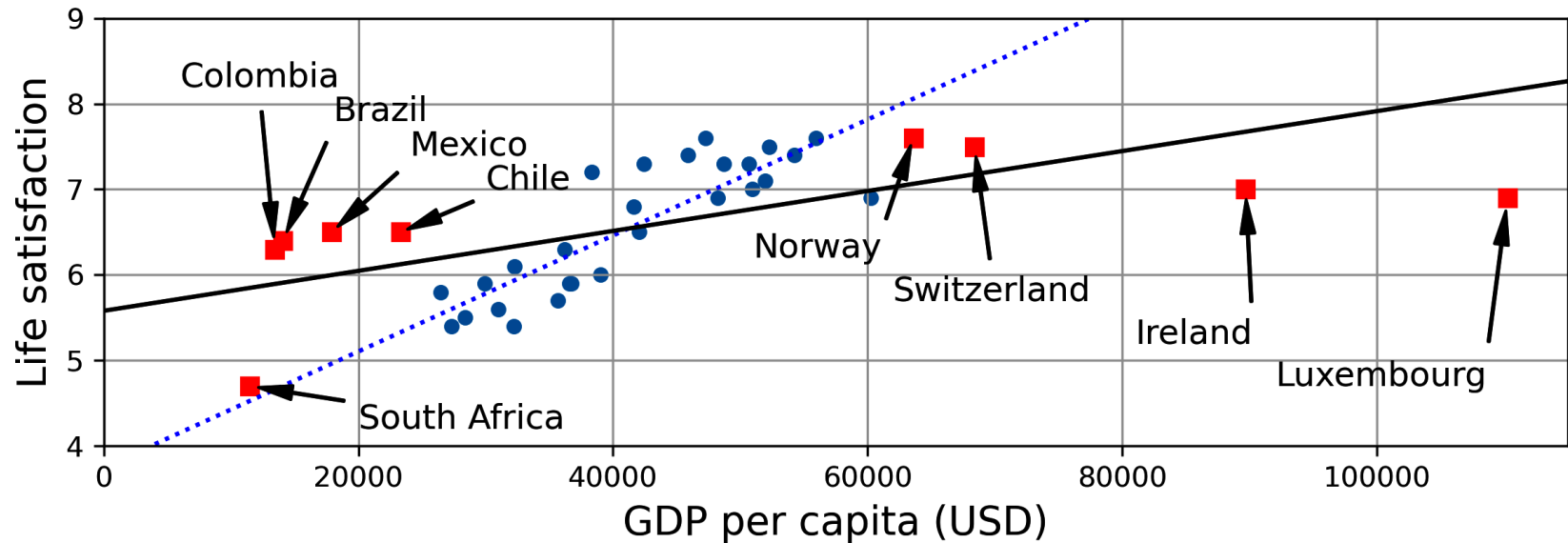
- In a **famous paper** published in 2001, Microsoft researchers Michele Banko and Eric Brill showed that very different machine learning algorithms, including fairly simple ones, performed almost identically well on a complex problem of natural language disambiguation once they were given enough data
- Authors: “these results suggest that we may want to reconsider the tradeoff between spending time and money on algorithm development versus spending it on corpus development”.
- Supported by Peter Norvig et al. in a paper titled “**The Unreasonable Effectiveness of Data**”, published in 2009.
- Small and medium-sized datasets are still very common, and it is not always easy or cheap to get extra training data—so don’t abandon algorithms just yet.

Disambiguation = (For example, knowing whether to write “to”, “two”, or “too”, depending on the context)

Nonrepresentative Training Data

- Training data should be representative of new cases you want to generalize to.
- For example, the set of countries used for training the linear model was not perfectly representative
- Why: it did not contain any country with a GDP per capita lower than \$23,500 or higher than \$62,500.

With updated data



Representative Dataset

- This is often harder than it sounds:
- Small Sample → **sampling noise** (i.e., nonrepresentative data as a result of chance),
- Large sample but sampling method is flawed → **sampling bias**
- Example: **Election poll**

Poor-Quality Data

- Training data is full of errors, outliers, and noise
 - Difficult to detect the underlying
 - System is less likely to perform well.
- TODO:
 - spend time cleaning up your training data.

Most data scientists spend a significant part of their time doing just Data Cleanup.

Example

- Outlier:
 - Discard them or try to fix the errors manually
- Missing Features:
 - Ignore the feature
 - Ignore those instances
 - Fill in the missing values (median)
 - Train two models (with feature; w/o feature)

Irrelevant Features

- Garbage in → Garbage out
- Training data should contain enough relevant features and not too many irrelevant ones
- Feature engineering:
 - Coming up with a good set of features to train on
 - A critical part of the success of a machine learning project

Steps involved in Feature engineering

- Feature selection

- Selecting the most useful features to train on among existing features

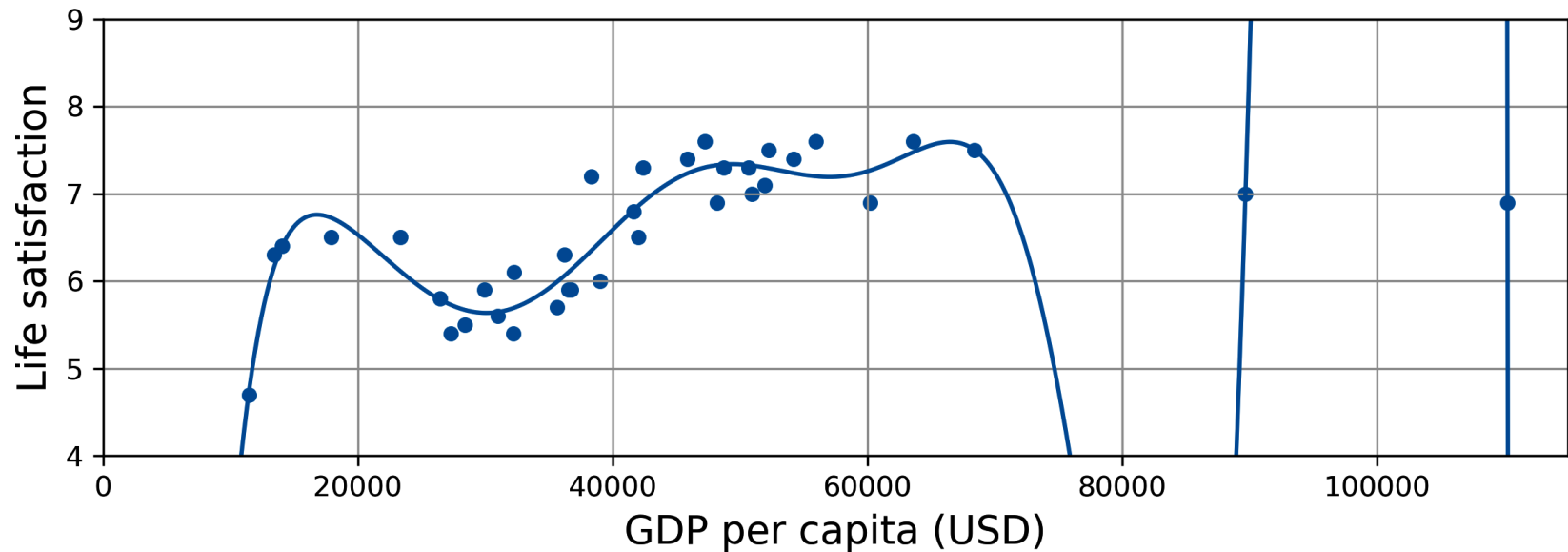
- Feature extraction

- Combining existing features to produce a more useful one
- Dimensionality reduction algorithms can help)

- Creating new features by gathering new data

Overfitting the Training Data

- The model performs well on the training data, but it does not generalize well.



Example

Complex models such as deep neural networks can detect subtle patterns in the data.

Training set is noisy, or small → **Sampling Noise**

- Model is likely to detect patterns in the noise itself.

These patterns will not generalize to new instances.

Example: ALL countries with a **w** in their name have a life satisfaction greater than **7**

- New Zealand (7.3)
- Norway (7.6)
- Sweden (7.3) and
- Switzerland (7.5)

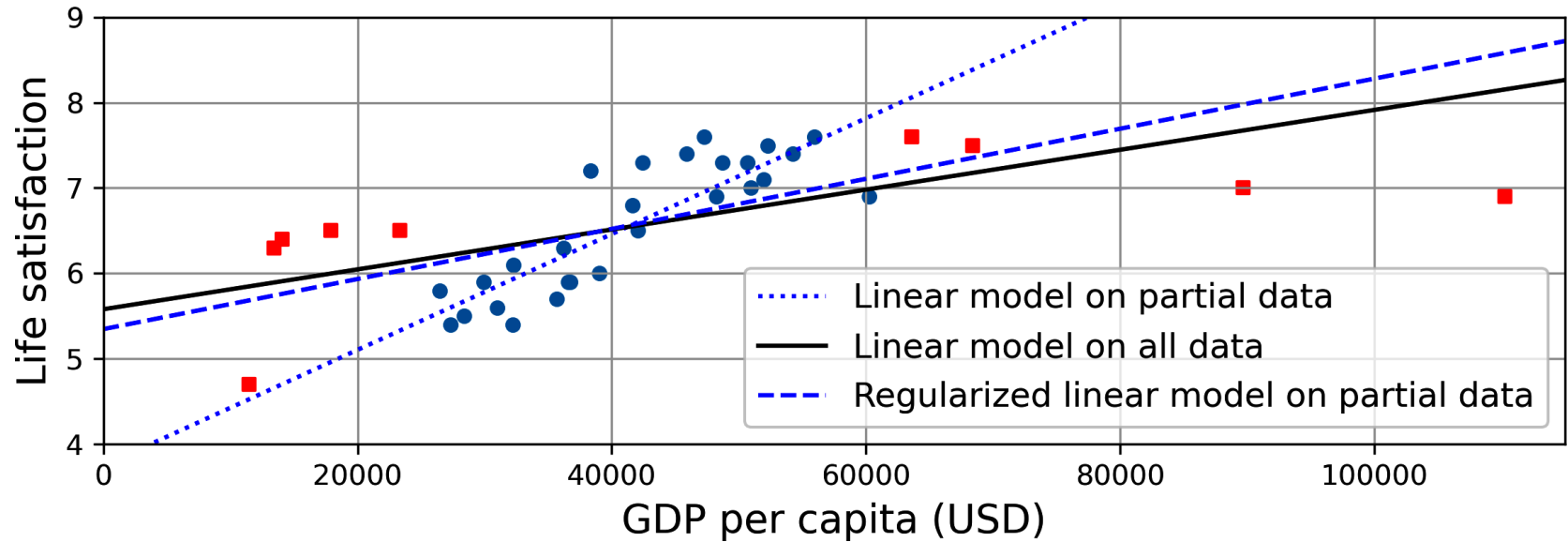
Model is complex vs. Data is small/noisy

Solution to Overfitting

- Simplify the model by
 - Selecting one with fewer parameters
 - e.g., a linear rather than a high-degree polynomial model
 - Reducing the number of attributes in the training data, or
 - By constraining the model
- Gather more training data.
- Reduce the noise in the training data (e.g., fix data errors and remove outliers).

Model is complex vs. Data is small/noisy

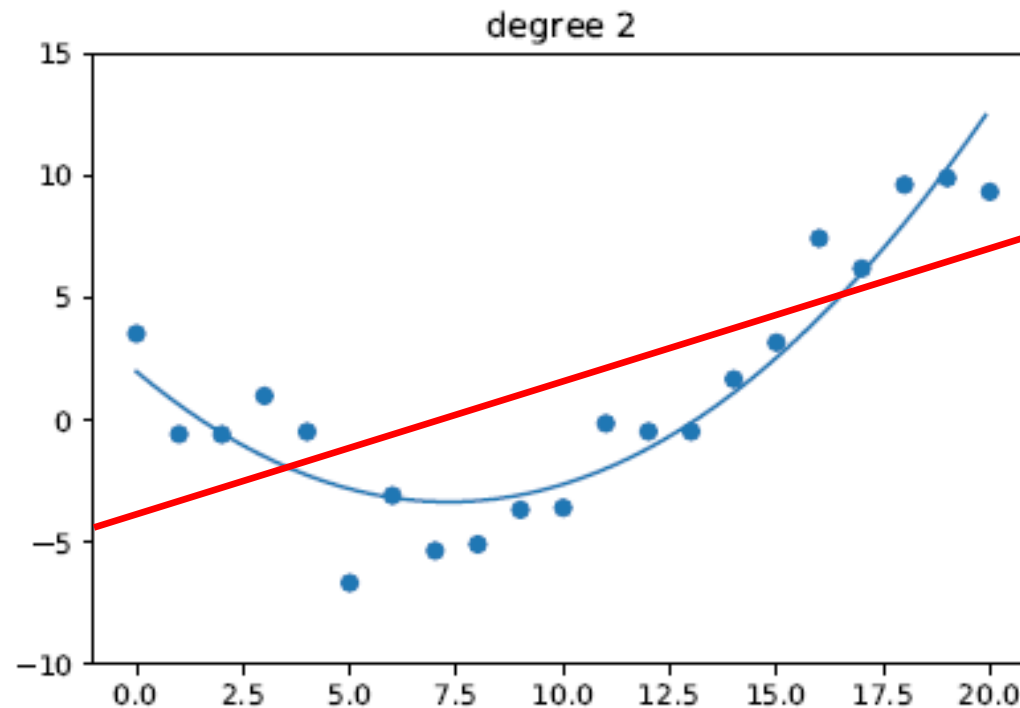
Regularization



Underfitting the Training Data

- Underfitting is the opposite of overfitting

Model is too simple for the Data



Solution

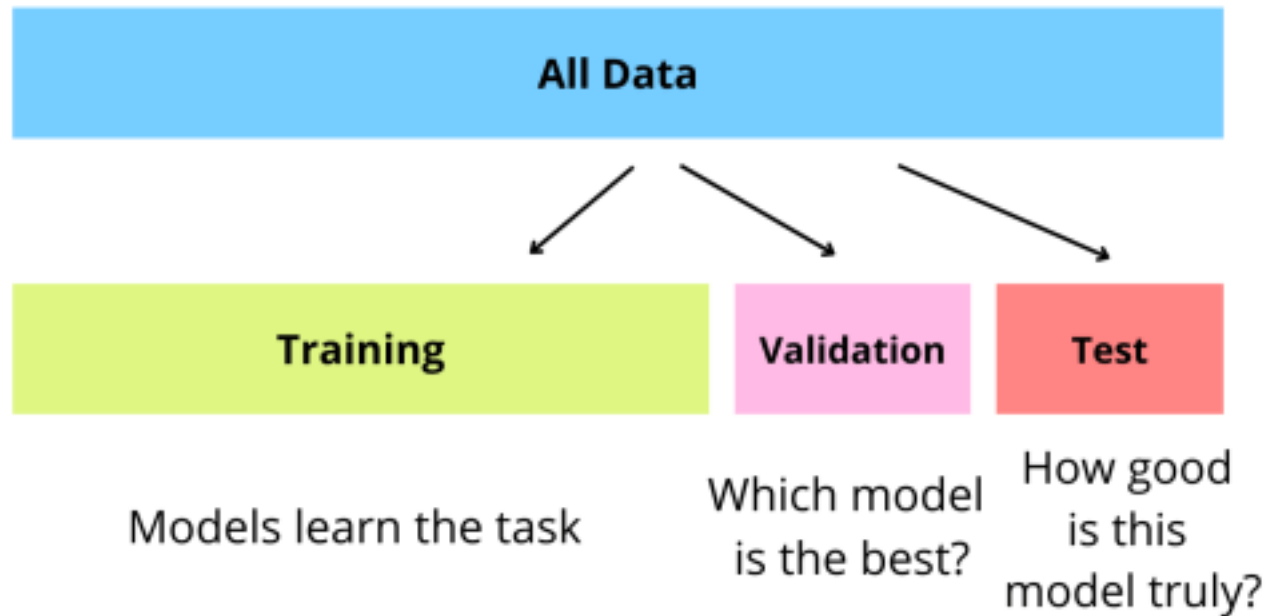
- Select a more powerful model, with more parameters.
- Feed better features to the learning algorithm (feature engineering).
- Reduce the constraints on the model (for example by reducing the regularization hyperparameter).

Training and Test Set

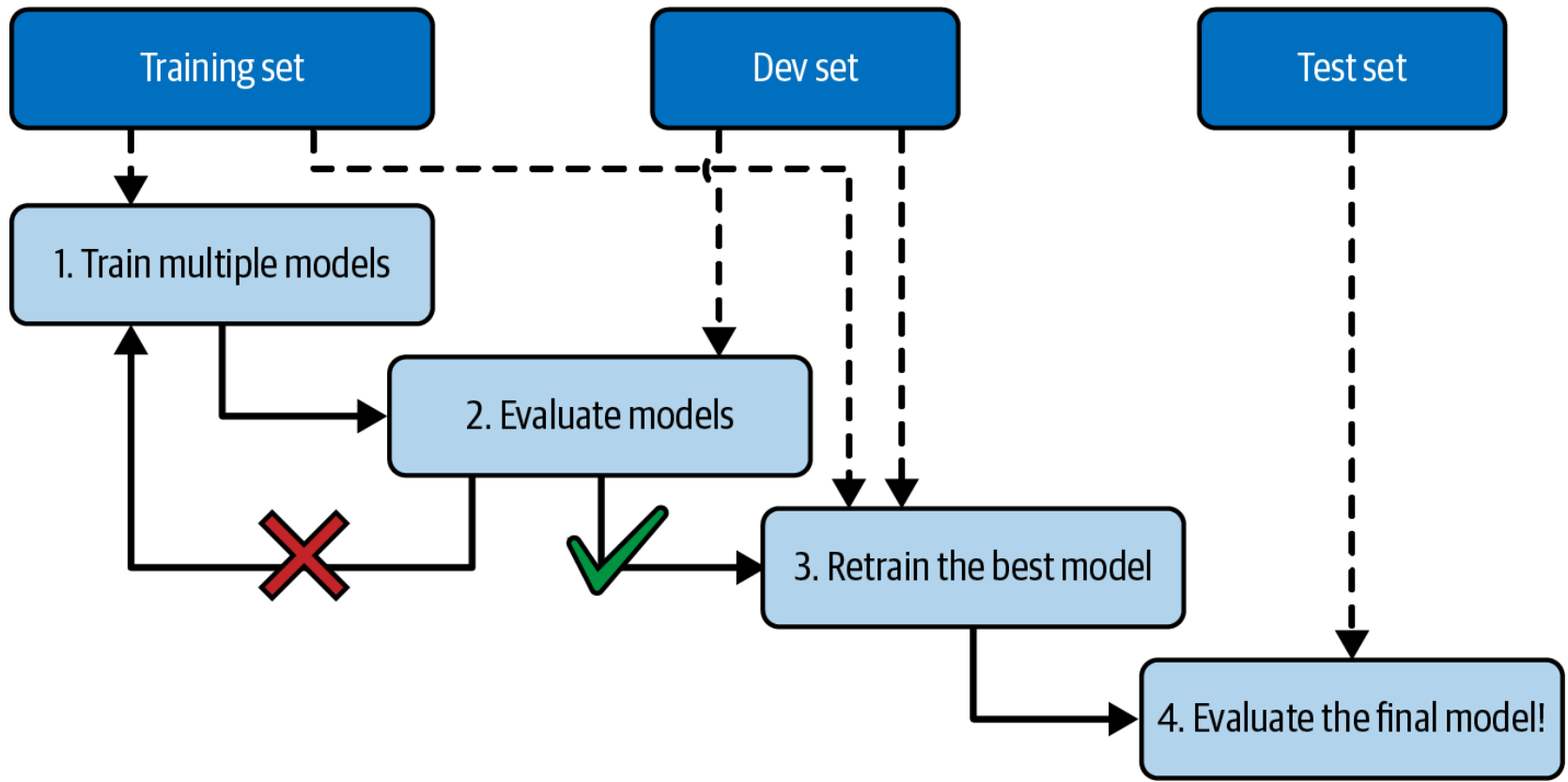
- Split your data into two sets
 - Training set
 - Test set.
- Train your model using the training set,
- Test your model with the text set
- The error rate on new cases is called the generalization error (or out-of-sample error)
- By evaluating your model on the test set, you get an estimate of this error.
- This value tells you how well your model will perform on instances it has never seen before

Common to use 80% of the data for training and hold out 20% for testing.
But it depends on the problem and data.

Data Validation



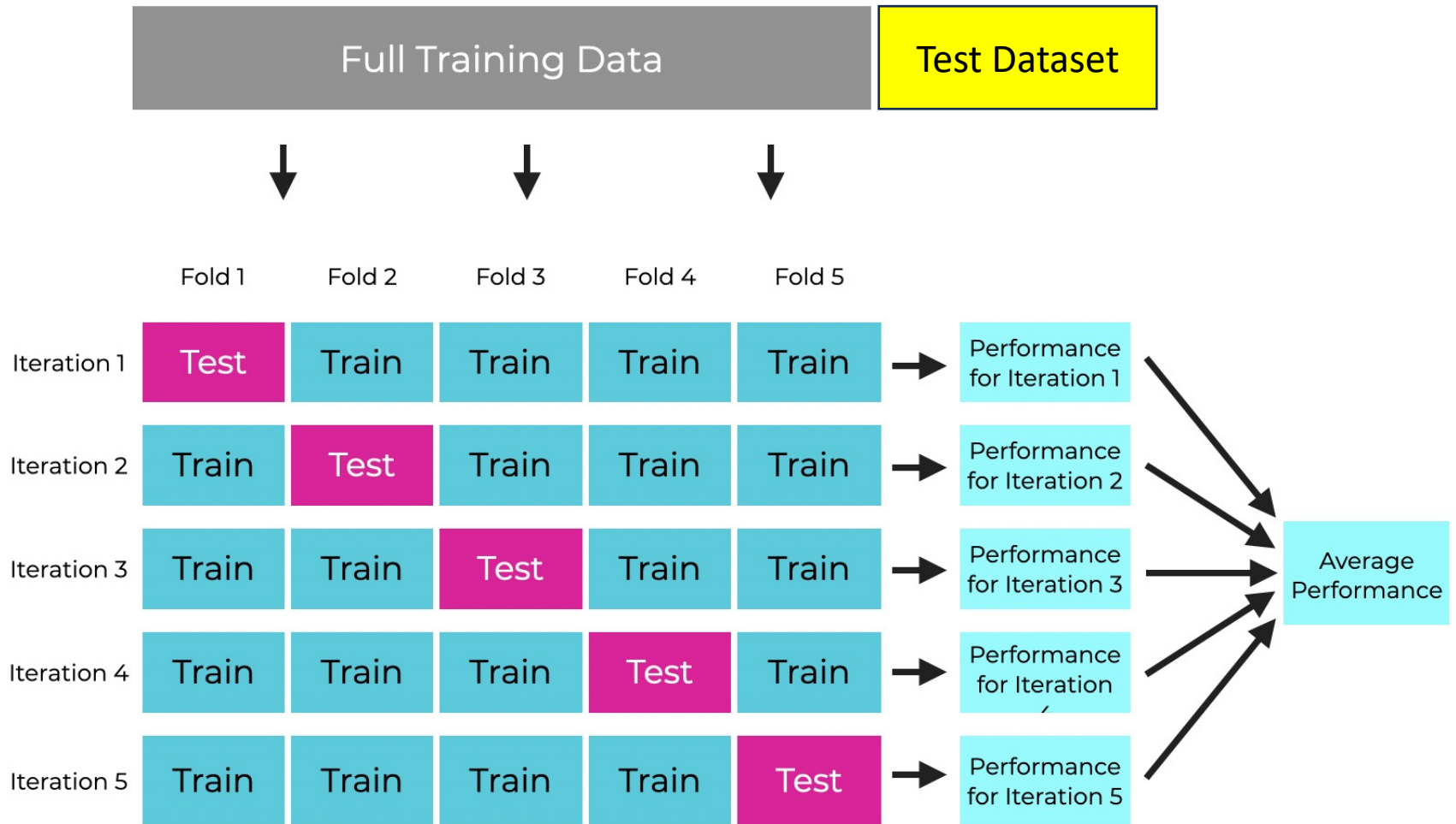
Data Validation



Cross Validation

- Small Validation Set → Imprecise Model Eval.
- Large Validation Set → Small Training Set
- Solution: Cross-validation

Cross validation



Note: We won't touch Test Dataset during training or validation

No Free Lunch Theorem

Theorem: If we make absolutely no assumption about the data, then there is no reason to prefer one model over any other.

Paper: David Wolpert, "*The Lack of A Priori Distinctions Between Learning Algorithms*", Neural Computation 8, no. 7 (1996): 1341–1390.

- There is no model that is a priori guaranteed to work better
- The only way to know for sure which model is best is to evaluate them all.
- Since this is not possible, in practice you make some reasonable assumptions about the data and evaluate only a few reasonable models.
- For example:
 - For simple tasks one may evaluate linear models with various levels of regularization, and
 - For a complex problem one may evaluate various neural networks.

Summary

- What is Machine Learning?
- Types of ML systems:
Supervised, unsupervised, or RL; batch or online, instance-based or model-based.
- In an ML project, training set is feed to a learning algorithm.
- If the algorithm is model-based, it tunes some parameters to fit the model to the training set
- If the algorithm is instance-based, it uses similarity measure to compare them to the learned instances.

Summary...

- The system will not perform well if your training set is
 - too small
 - not representative
 - noisy
 - polluted with irrelevant features (garbage in, garbage out).
- The model should be
 - neither too simple nor too complex
- Test set and Validation set
- Cross Validation
- No Free Lunch Theorem

Thank you all!

Acknowledgement

- Slides are based on Textbooks
- Slides found online Many slides including slides by:
 - Saw Shier Nee, Dept. of AI, University of Malaya
 - Andrew Ng, DeepLearning.AI